

Backend Summary

Study Group Finder API

Technical Reference Document

What It Is

A **REST API** built with Node.js and Express that powers the Study Group Finder application. It handles user authentication and study session management, storing all data in a MySQL database.

Property	Value
Runtime	Node.js (CommonJS modules)
Framework	Express 5
Database	MySQL via <code>mysql2/promise</code> connection pool
Port	5003 (configurable via <code>.env</code>)

How It Starts

```
npm run dev -> nodemon src/server.js
```

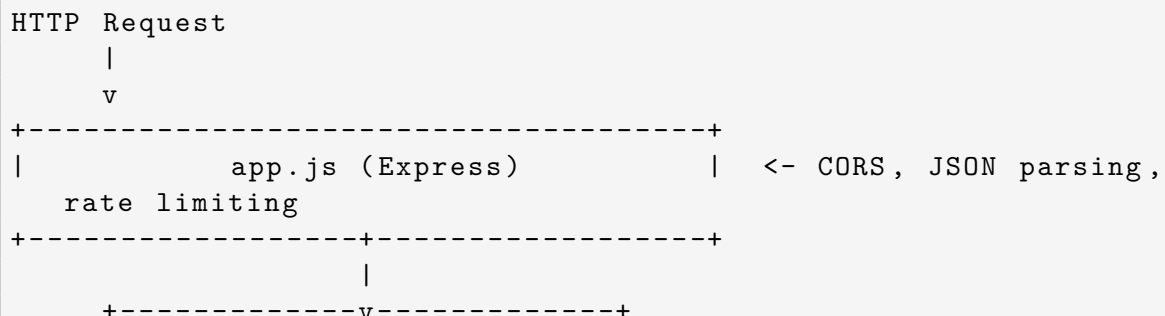
`server.js` does three things in order:

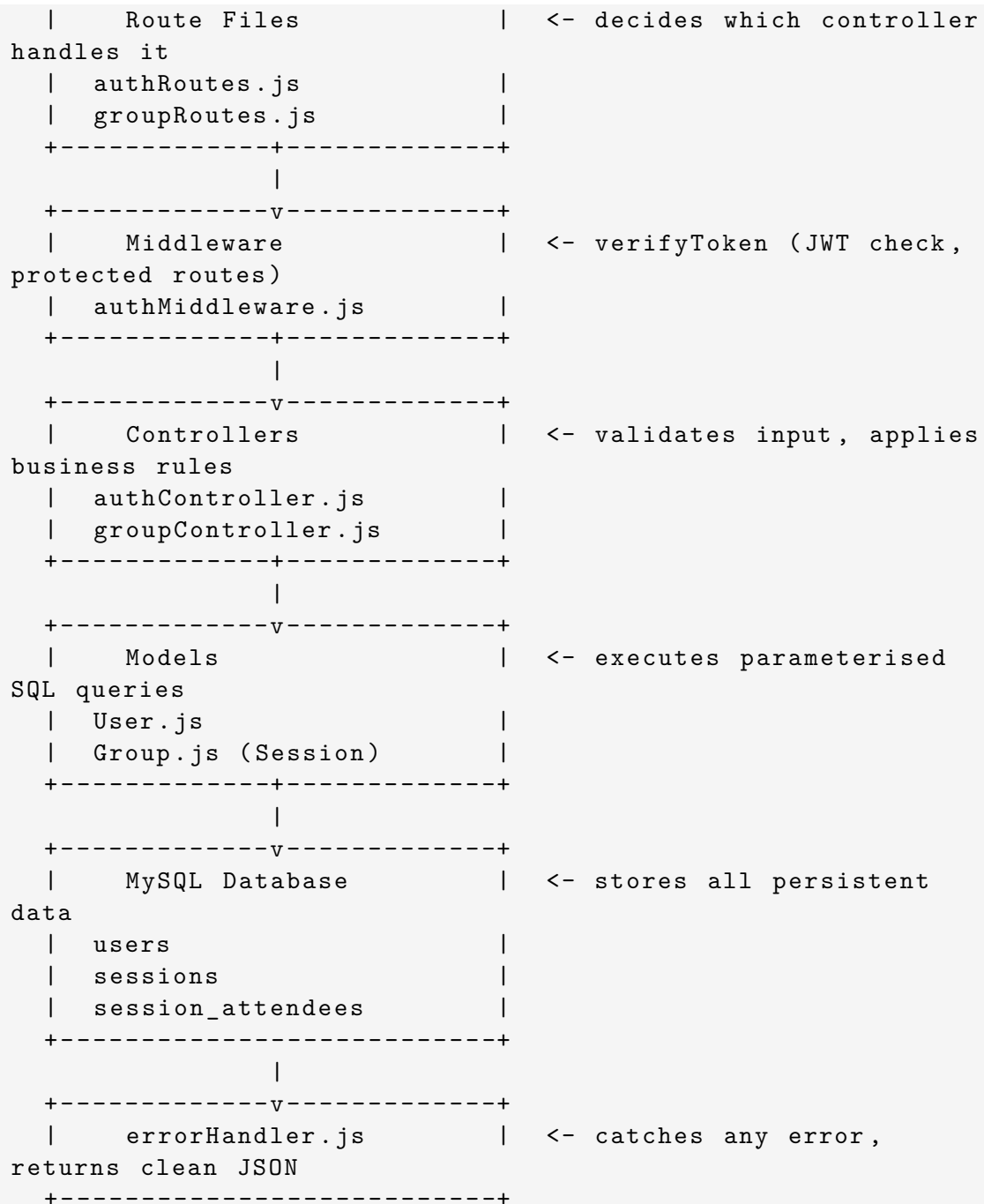
1. **Loads `.env`** — all secrets and config (DB credentials, JWT secret, port) come from environment variables, never hardcoded.
2. **Opens the DB connection pool** — `config/db.js` is required, which immediately tests the MySQL connection.
3. **Runs `initDB()`** — creates the three database tables if they do not exist yet, then starts listening.

Note: If MySQL is unreachable, the server **refuses to start** and exits with code 1 rather than crashing mid-request.

Architecture — The Layers

Every HTTP request travels through these layers in order:





The Database (3 Tables)

users

Stores registered accounts.

Column	Description
id	Primary key
username	Display name
email	Unique email address
password	bcrypt hash (never plain text)
created_at	Timestamp of registration

sessions

Stores every study session ever created.

Column	Description
id	Primary key
tag	Category tag
title	Session title
location	Text description of location
time	Scheduled time
about	Session description
total_spots	Maximum number of attendees
lat, lng	Geographic coordinates
color	UI display colour
host_id	FK → users.id (cascade delete)

session_attendees

Join table that tracks who joined which session.

Detail	Value
Primary key	Composite (session_id, user_id)
Duplicate joins	Prevented by composite PK
On session delete	Cascade removes all attendee rows

All API Endpoints

Auth — /api/auth

Method	Endpoint	Auth	What it does
POST	/api/auth/register	No	Create a new account
POST	/api/auth/login	No	Log in, receive a JWT token
POST	/api/auth/logout	No	Stateless logout (client drops token)

Sessions — /api/sessions

Method	Endpoint	Auth	What it does
GET	/api/sessions	No	Get all sessions (public)
GET	/api/sessions/:id	No	Get one session + attendees
POST	/api/sessions	JWT	Create a new session
POST	/api/sessions/:id/join	JWT	Join a session
DELETE	/api/sessions/:id/leave	JWT	Leave a session
DELETE	/api/sessions/:id	JWT	Delete a session (host only)

Utility

Method	Endpoint	Auth	What it does
GET	/api/health	No	Check the server is running
GET	/	No	API welcome message

How Each Operation Works

Register

1. Joi validates the body (`username`, `email`, `password`).
2. Checks the DB for a duplicate email — rejects if found.
3. `bcrypt` hashes the password (salt rounds = 12).
4. Inserts the user into MySQL.
5. Returns the new user (no password in the response).

```
POST /api/auth/register
{ "username": "Nedim", "email": "nedim@unic.ac.cy", "password":
  "secret123" }

-> 201 { "data": { "id": 1, "username": "Nedim", "email": "..."} }
```

Login

1. Joi validates the body.
2. Finds the user by email in the DB.
3. `bcrypt.compare()` checks the submitted password against the stored hash.
4. On success, signs a JWT containing `{id, email, username}`, valid for 24 hours.
5. Returns the token — the client stores this and sends it on every protected request.

```
POST /api/auth/login
{ "email": "nedim@unic.ac.cy", "password": "secret123" }
```

```
-> 200 { "data": { "token": "eyJ...", "user": { "id": 1, ... } } }
```

Protected Request (e.g. Create Session)

Every protected route runs `verifyToken` middleware first:

1. Checks for Authorization: `Bearer <token>` header.
2. `jwt.verify()` decodes and validates the token against `JWT_SECRET`.
3. Attaches decoded user (`{id, email, username}`) to `req.user`.
4. Calls `next()` — the controller runs.

Create Session

1. Joi validates the body (`tag, title, location, time, totalSpots`, etc.).
2. Inserts into the `sessions` table with `host_id = req.user.id`.
3. Returns the full session object.

Join Session

1. Verifies the session exists.
2. Rejects if the user is the host (they are already the creator).
3. Calculates `spots left = total_spots - COUNT(attendees)` — rejects if full.
4. `INSERT IGNORE` into `session_attendees` (idempotent — joining twice is safe).
5. Returns the updated session with its attendees list.

Delete Session

1. Runs `DELETE FROM sessions WHERE id = ? AND host_id = req.user.id`.
2. If `affectedRows = 0`, the session either does not exist or the user is not the host — returns 403.
3. Cascade delete in MySQL removes all `session_attendees` rows automatically.

Security Measures

Measure	Implementation
Password hashing	<code>bcryptjs</code> with 12 salt rounds — passwords never stored or returned as plain text
JWT authentication	Signed with <code>JWT_SECRET</code> from <code>.env</code> , 24-hour expiry
Rate limiting	Login endpoint: max 10 attempts per 15 minutes (blocks brute-force)
Input validation	Every endpoint validates its body with Joi before touching the DB
SQL injection prevention	All queries use parameterised <code>?</code> placeholders — no string concatenation
Secrets in env	All credentials live in <code>.env</code> , never in source code
Owner-gated deletes	<code>host_id</code> check enforced in SQL, not just in application code

Project File Map

```
src/
|-- server.js          <- entry point: load env, init DB,
    start server
|-- app.js             <- express setup: middleware, routes,
    error handler
|
|-- config/
|   |-- db.js          <- MySQL connection pool
|   '-- initDB.js      <- CREATE TABLE IF NOT EXISTS
                        (users, sessions,
                        session_attendees)
|
|-- routes/
|   |-- authRoutes.js  <- POST /register /login /logout
|   '-- groupRoutes.js <- GET/POST/DELETE /api/sessions (+
    join, leave)
|
|-- controllers/
|   |-- authController.js <- register, login, logout logic
|   '-- groupController.js <- getSession, createSession,
    joinSession,
                        leaveSession, deleteSession
|
|-- models/
|   |-- User.js        <- findByEmail, findById, createUser
|   '-- Group.js       <- getAll, getById, create, join,
    leave,
                        delete, isJoined, spotsLeft
|-- middleware/
```

```
|-- authMiddleware.js  <- verifyToken (JWT guard for
protected routes)
'-- errorHandler.js    <- global error catcher -> clean JSON
error response
```

What Is Not Yet Done

Feature	Status
User profile update (PUT /api/users/me)	Not implemented
Session update (PUT /api/sessions/:id)	Not implemented
Pagination on GET /api/sessions	Not implemented
Search / filter sessions by tag or location	Not implemented
Refresh tokens (renew JWT without re-login)	Not implemented
Front-end ↔ back-end wiring	Not done (front-end uses localStorage)

Generated from backend_summary.md — Study Group Finder API Reference